

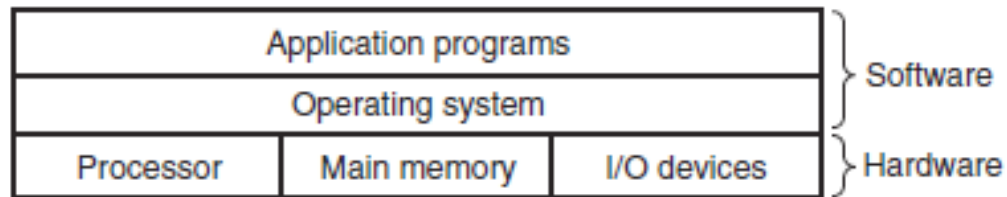
Computer Organization and Architecture

Contents

- 1.1 Computer Systems
- 1.2 Programs Are Translated by Other Programs into Different Forms
- 1.3 Hardware Organization of a System
- 1.4 Concurrency and Parallelism
- 1.5 The Evolution of The Intel x86 Architecture
- 1.6 Computer Architecture
- 1.7 Instruction Set Architecture (ISA)

Computer Systems

- A computer system consists of hardware and system software that work together to run application programs.



From R.E. Bryant and D.R. O'Halloron, Computer Systems: A Programmer's Perspective, 2nd edition, 2011, Chapter 1.

Programs Are Translated by Other Programs into Different Forms

- The hello program begin life as a high-level C program because it can be read and understood by human.

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello, world\n");
6  }
```

code/intro/hello.c

Figure 1.1 The hello program.

From R.E. Bryant and D.R. O'Halloron, Computer Systems: A Programmer's Perspective, 2nd edition, 2011, Chapter 1.

Programs Are Translated by Other Programs into Different Forms

- The hello program begin life as a high-level C program because it can be read and understood by human.

#	i	n	c	l	u	d	e	<sp>	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	<sp>	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	<sp>	<sp>	<sp>	<sp>	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	<sp>	w	o	r	l	d	\	n	"	)	;	\n	}
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	125

Figure 1.2 The ASCII text representation of `hello.c`.

From R.E. Bryant and D.R. O'Halloron, *Computer Systems: A Programmer's Perspective*, 2nd edition, 2011, Chapter 1.

Programs Are Translated by Other Programs into Different Forms

- To run `hello.c` on the system, it must be translated by other programs into a sequence of low-level machine-language instructions.
- These instructions are then packaged in a form called an executable object program.

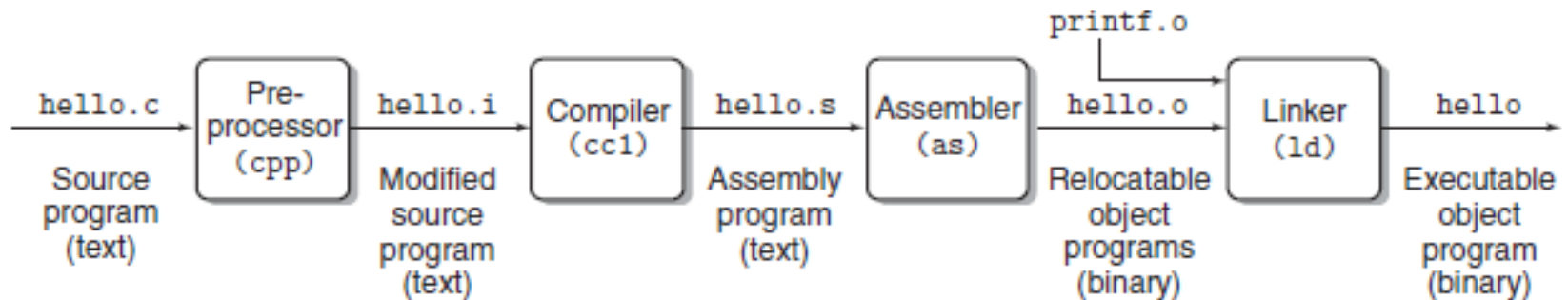


Figure 1.3 The compilation system.

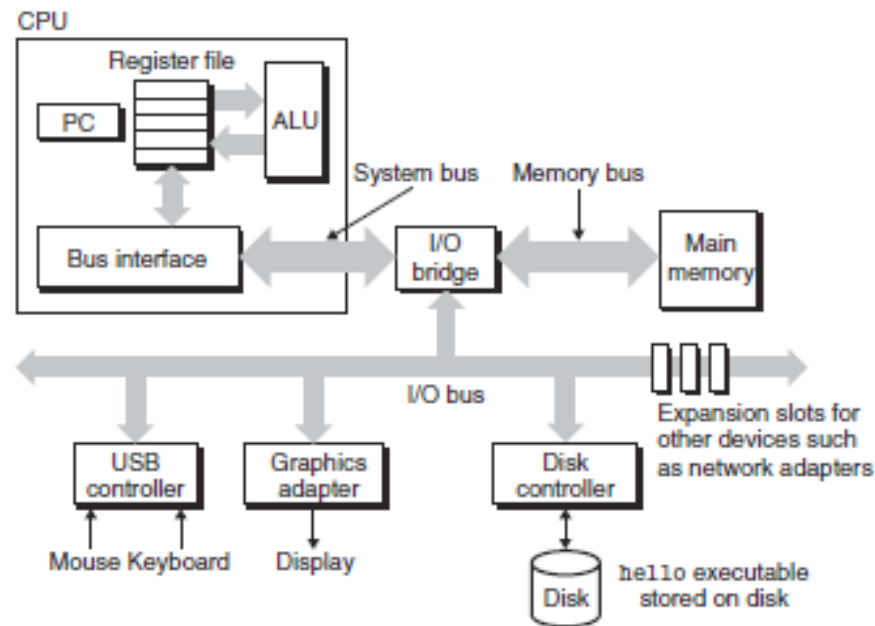
From R.E. Bryant and D.R. O'Halloron, *Computer Systems: A Programmer's Perspective*, 2nd edition, 2011, Chapter 1.

Hardware Organization of a System

- To understand what happens to our hello program when we run it, we need to understand the hardware organization of a typical system.

Figure 1.4

Hardware organization of a typical system. CPU: Central Processing Unit, ALU: Arithmetic/Logic Unit, PC: Program counter, USB: Universal Serial Bus.



From R.E. Bryant and D.R. O'Halloron, Computer Systems: A Programmer's Perspective, 2nd edition, 2011, Chapter 1.

Hardware Organization of a System

- The central processing unit (CPU), or simply processor, is the engine that interprets (or executes) instructions stored in main memory.
- From the time that power is applied to the system until the time that the power is shut off, a processor repeatedly executes the instruction.
- A processor appears to operate according to a very simple instruction execution model, defined by its instruction set architecture.

Hardware Organization of a System

- The main memory is a temporary storage device that holds both a program and the data it manipulates while the processor is executing the program.
- Physically, main memory consists of a collection of dynamic random access memory (DRAM) chips.
- Logically, memory is organized as a linear array of bytes, each with its own unique address (array index) starting at zero.

Concurrency and Parallelism

- Throughout the history of digital computers, two demands have been constant forces in driving improvements:
 - We want them to do more, and
 - We want them to run faster.
- We use the term **concurrency** to refer to the general concept of a system with multiple, simultaneous activities, and
- The term **parallelism** to refer to the use of concurrency to make a system run faster.

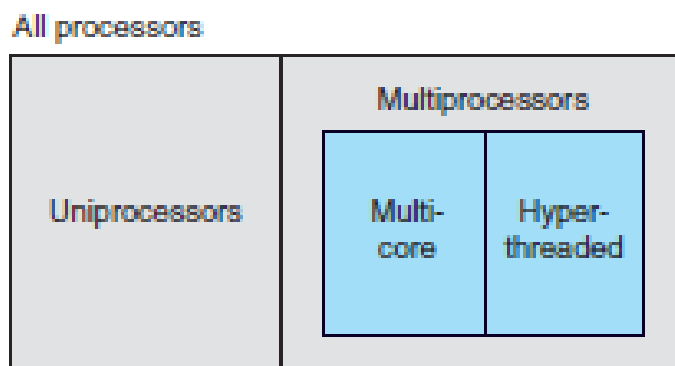
Concurrency and Parallelism

- Parallelism can be exploited at multiple levels of abstraction in a computer system.
- We highlight three levels here, working from the highest to the lowest level in the system hierarchy.
 1. Thread-Level Concurrency
 2. Instruction-Level Parallelism
 3. Single-Instruction, Multiple-Data (SIMD) Parallelism

Thread-Level Concurrency

- Building on the process abstraction, we are able to devise systems where multiple programs execute at the same time, leading to concurrency.

Figure 1.16
Categorizing different processor configurations. Multiprocessors are becoming prevalent with the advent of multi-core processors and hyperthreading.



From R.E. Bryant and D.R. O'Halloron, Computer Systems: A Programmer's Perspective, 2nd edition, 2011, Chapter 1.

Thread-Level Concurrency

- Multi-core processors have several CPUs (referred to as “cores”) integrated onto a single integrated-circuit chip.
- Hyperthreading, sometimes called simultaneous multi-threading, is a technique that allows a single CPU to execute multiple flows of control.
- As an example, the Intel Corei7 processor can have each core executing two threads, and so a four-core system can actually execute eight threads in parallel.

Instruction-Level Parallelism

- Modern processors can execute multiple instructions at one time, a property known as instruction-level parallelism.
- Processors that can sustain execution rates faster than 1 instruction per cycle are known as superscalar processors.

Single-Instruction, Multiple-Data (SIMD) Parallelism

- Many modern processors have special hardware that allows a single instruction to cause multiple operations to be performed in parallel, a mode known as single-instruction, multiple-data (SIMD) parallelism.
- For example, recent generations of Intel and AMD processors have instructions that can add 8 pairs of single-precision floating-point numbers in parallel.

The Evolution of The Intel x86 Architecture

- The Intel processor line, colloquially referred to as x86, has followed a long evolutionary development.
- It started with one of the first single-chip 16-bit microprocessors, where many compromises had to be made due to the limited capabilities of integrated circuit technology at the time.
- Since then, it has grown to take advantage of technology improvements as well as to satisfy the demands for higher performance and for supporting more advanced operating systems.

The Evolution of The Intel x86 Architecture

- The list that follows shows some models of Intel processors and some of key features, especially those affecting machine-level programming.
- 8086 (1978, 29 K transistors) – One of the first single-chip, 16-bit microprocessors. The 8088, a variant of the 8086 with an 8-bit external bus, formed the heart of the original IBM personal computers. Intel introduced the 8087 floating-point coprocessor.
- 80286 (1982, 134 K transistors) – Added more (and now obsolete) addressing modes.
- i386 (1985, 275 K transistors) – Expand the architecture to 32 bits.
- i486 (1989, 1.2 M transistors) – Improved performance and integrated the floating-point unit onto the processor chip but did not significantly change the instruction set.

The Evolution of The Intel x86 Architecture

- Pentium (1993, 3.1 M transistors) – Improved performance but only added minor extensions to the instruction set.
- PentiumPro (1995, 5.5 M transistors) – Introduced a radically new processor design, internally known as the P6 microarchitecture. Added a class of “conditional move” instructions to the instruction set.
- Pentium/MMX (1997, 4.5 M transistors) – Added new class of instructions to the Pentium processor for manipulating vectors of integers.
- PentiumII (1997, 7 M transistors) – Continuation of the P6 microarchitecture.

The Evolution of The Intel x86 Architecture

- PentiumIII(1999, 8.2 M transistors) – Introduced SSE, a class of instructions for manipulating vectors of integer or floating-point data.
- Pentium4(2000, 42 M transistors) – Extended SSE to SSE2
- Pentium4E(2004, 125 M transistors) – Added hyperthreading, a method to run two programs simultaneously on a single processor, as well as EM64T, Intel's implementation of a 64-bit extension to IA32 developed by Advanced Micro Devices (AMD), which we refer to as x86-64.
- Core2(2006, 291 M transistors) – First multi-core Intel microprocessor, where multiple processors are implemented on a single chip. Did not support hyperthreading.

The Evolution of The Intel x86 Architecture

- Corei7, Nehalem (2008, 781 M transistors) – Incorporated both hyperthreading and multi-core, with the initial version supporting two executing programs on each core and up to four cores on each chip.
- Corei7, Sandy Bridge(2011, 1.17 G transistors) – Introduced AVX.
- Corei7, Haswell (2013, 1.4 G transistors) – Extended AVX to AVX2.

Computer Architecture

- Computer architecture refers to those attributes of a system visible to a programmer.
- A term that is often used interchangeably with computer architecture is instruction set architecture (ISA).
- Computer organization refers to the operational units and their interconnections that realize the architectural specifications.
- Example
 - Architectural design issue: a computer will have a multiply instruction whether or not.
 - Organizational issue: the multiply instruction will be implemented by a special multiply unit or by a mechanism that makes repeated use of the add unit of the system.

Computer Architecture

- The Von Neumann Model
- Non-Von Neumann Models
 - Harvard architecture
 - Modified Harvard architecture

Von Neumann Architecture

- Share instruction and data in the same memory. It can read either an instruction or a word of data at one time. This problem is called the Von Neumann bottleneck.

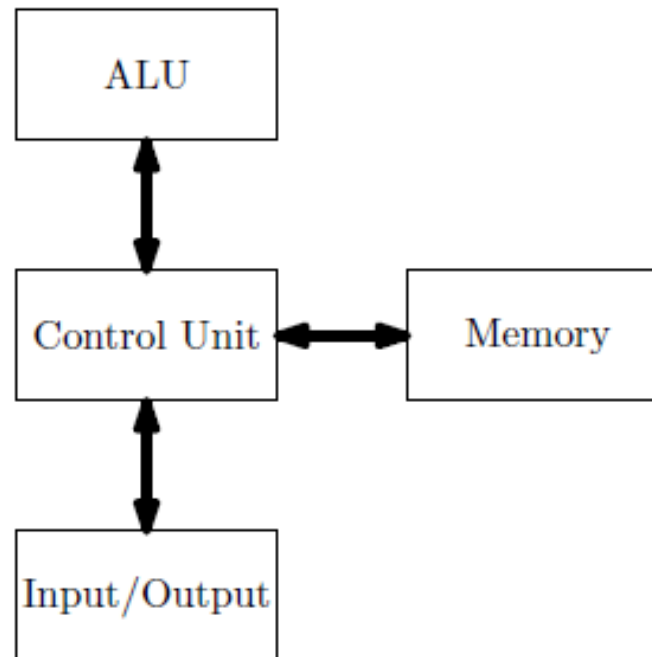


Figure 1.7: Von Neumann architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Harvard Architecture

- It is an architecture that physically separates storage for data and instructions.

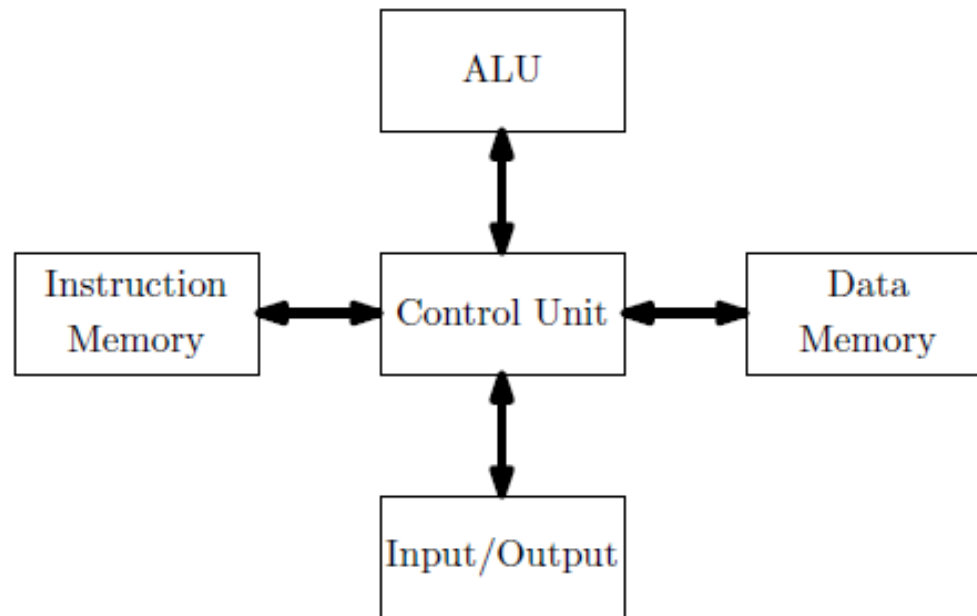


Figure 1.4: Harvard architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Modified Harvard Architecture

- Note: Cache is referred to a faster memory which stores copies of the contents of the main memory.

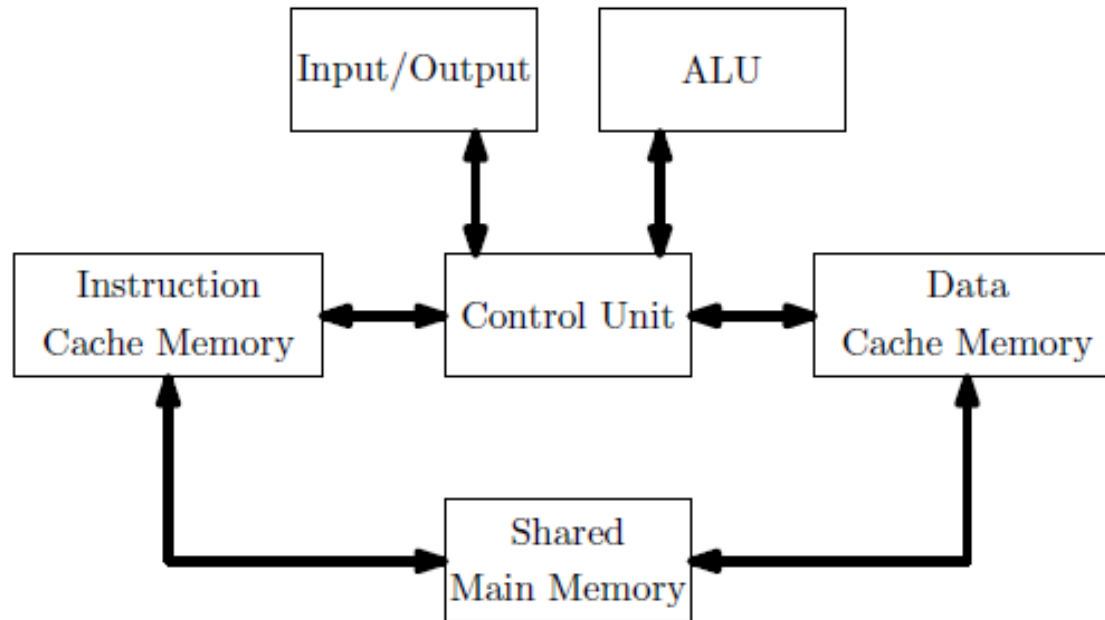


Figure 1.8: An Example of Modified Harvard Architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Instruction Set Architecture (ISA)

- The ISA is the interface between all the software that runs on the machine and the hardware that executes it.
- An ISA typically defines
 1. instructions to perform various computing operations
 2. temporary storage to store values in the processor
 3. addressing modes specifying memory locations.
- Examples of ISAs are
 - IA32 (Intel architecture, 32-bit) or i386 is the 32-bit version of the ISA manufactured by Intel.
 - x86-64 is the 64-bit extension of the IA-32. It is first introduced by AMD. Therefore, it is called 'AMD64'.
 - ARM (Advanced RISC Machine) is an ISA developed by ARM Holding. It is widely used in the microprocessors for smart phones.

Instructions

- An Instruction is used to refer to a primitive computation operation performed by the CPU.
- For example, addition, subtraction, data copy, etc.
- An instruction consists of (1) an operation (2) a number of operands.

ISA	Machine code/Assembly code
ARM	1110 0010 1000 0110 0110 0000 0000 0001 = add r6, r6, #1
IA-32	0100 0000 = inc %eax

Note: r6 and eax are called register. They are located in the temporary storage.

Temporary Storage

- Temporary storage (Temp) is an area in the processor.
- It is used by the processor to manage the instruction execution and it is used by programmers to temporarily hold values in the program.
- We can classify ISAs based on the structure of the temporary storage.
 1. Accumulator architecture
 2. Stack architecture
 3. General-purpose register architecture

Accumulator Architecture

- Accumulator architecture uses an accumulator register.
- All the operations are performed against the accumulator.

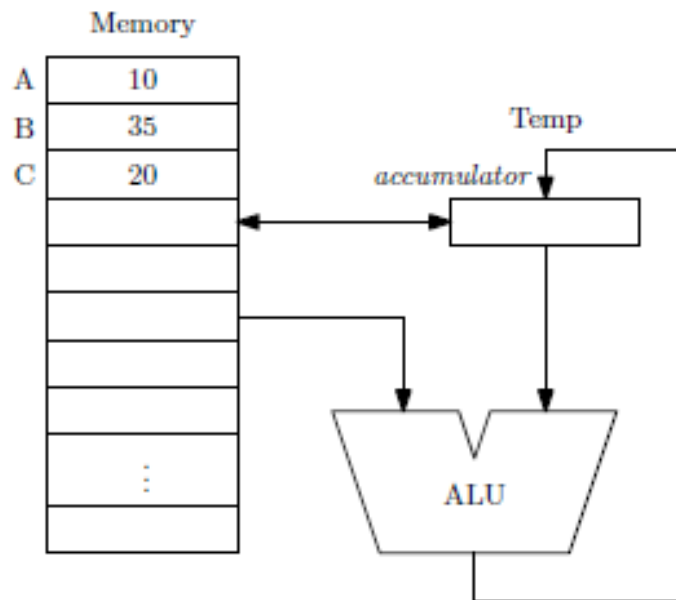


Figure 1.9: Accumulator architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Accumulator Architecture

- Example 1.1 An ISA with the accumulator architecture has four types of instructions.
 1. load M = to read a value of location M in the memory to the accumulator.
 2. store M = to write the accumulator value to the location M in the main memory.
 3. add M = to add a value of location M and the accumulator value and store the result in the accumulator.
 4. sub M = to subtract a value of location M from the accumulator value and store the result in the accumulator.

Accumulator Architecture

- An expression $C = A + B$ where A, B, and C are memory locations can be translated into assembly instruction as:

load A
add B
store C

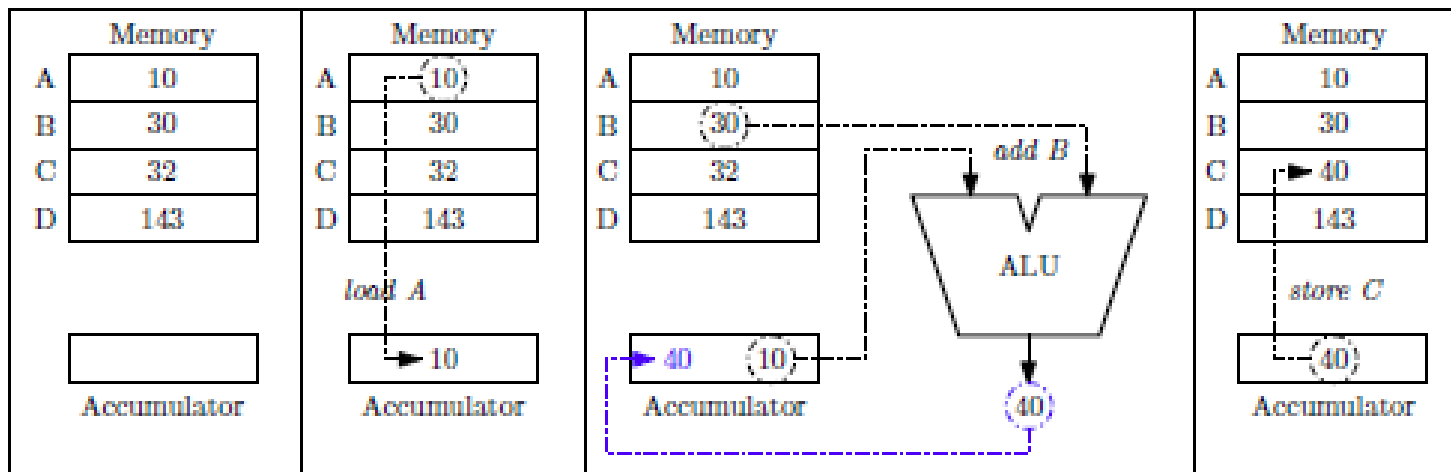


Figure 1.10: Execute $C = A + B$ in the Accumulator architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

In-class Exercise

- Exercise 1.3 Translate the following expression into assembly language.

$$A = (A+B)-(C+D)-E$$

Here, A, B, C, D, and E are memory locations.

Stack Architecture

- Stack architecture use a stack to execute instructions, and the operands are found on top of the stack.
- It provides push and pop operations to access data in the memory.
- The push operation loads the data at a specified address from the memory and append to the stack on the top of stack.
- The pop operation saves the data on the top of stack to a specified address.
- This architecture can be found in Java Virtual Machine.

Stack Architecture

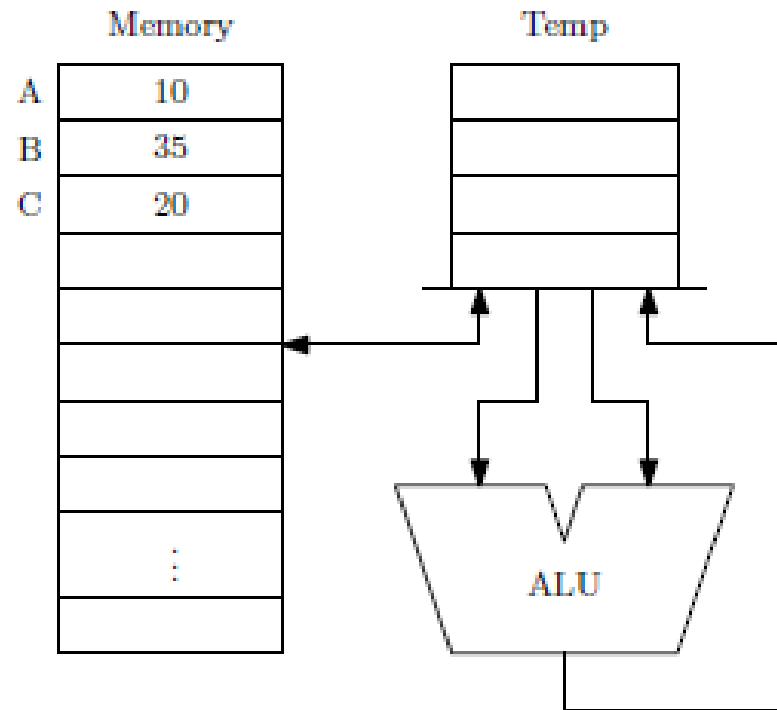


Figure 1.11: Stack architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Stack Architecture

- Example 1.2 An ISA with the stack architecture has four types of instructions.
 1. push M = to read a value of location M in the memory and push on to the top of stack.
 2. pop M = to remove a value on the top of the stack, and write it to the location M in the main memory.
 3. add = to remove two values on the top of the stack, add them, and push the result to the stack.
 4. sub = to remove two values on the top of the stack, subtract the first value from the second one, and push the result to the stack.

Stack Architecture

- An expression $C = A + B$ can be translated into assembly instruction as:

```
push A  
push B  
add  
pop C
```

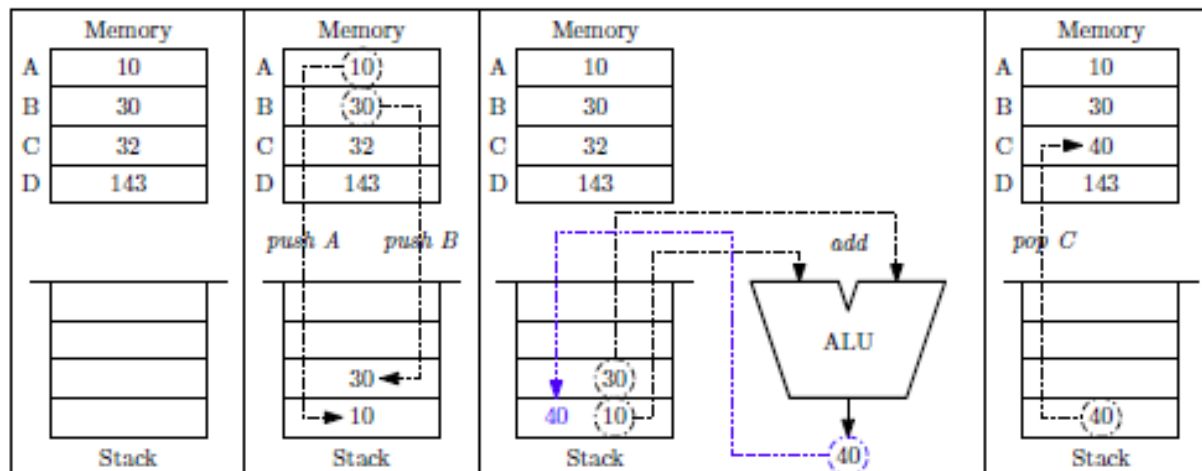


Figure 1.12: Execute $C = A + B$ in the Stack architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

In-class Exercise

- Exercise 1.4 Translate the following expression into assembly language.

$$A = (A+B)-(C+D)-E$$

Here, A, B, C, D, and E are memory locations.

General-Purpose Register Architecture

- General-purpose register architecture use sets of general-purpose registers.
- Each register can be referred in the code by its name.
- This is the most common architecture for modern processors.
- This architecture can be separated into two subtypes i.e.
 1. Register-register architecture
 2. Register-memory architecture

Register-Register Architecture

- Register-register architecture allows only the registers to be the operands of the instructions.
- It provides two special instructions for accessing data in the memory i.e. load and store.
- This architecture can be found in ARM processors.

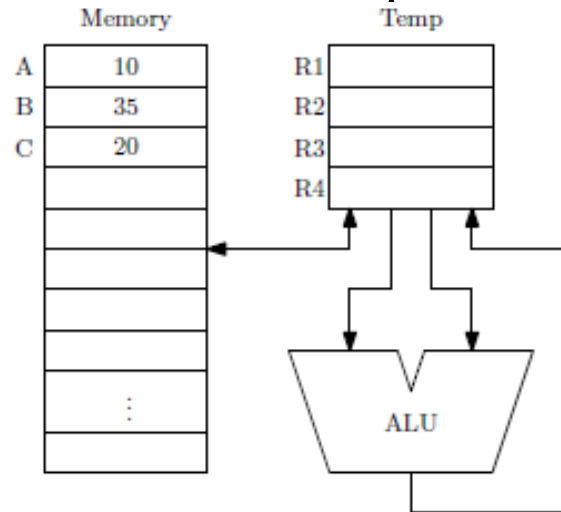


Figure 1.13: Register-Register architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Register-Register Architecture

- Example 1.3 An ISA with the register-register architecture has four types of instructions.
 1. load M,R = to read a value of location M and save it to the register R.
 2. store R,M = to write the value of register R to the location M in the main memory.
 3. add R1,R2,R3 = to read the value of register R1 and R2, add them, and save the result to register R3.
 4. sub R1,R2,R3 = to read the value of register R1 and R2, subtract R1 from R2, and save the result to register R3.

Register-Register Architecture

- Assume that there are 4 registers: R1,R2,R3 and R4. An expression $C=A+B$ can be translated into assembly instruction as:

```
load A,R1  
load B,R2  
add R1,R2,R3  
store R3,C
```

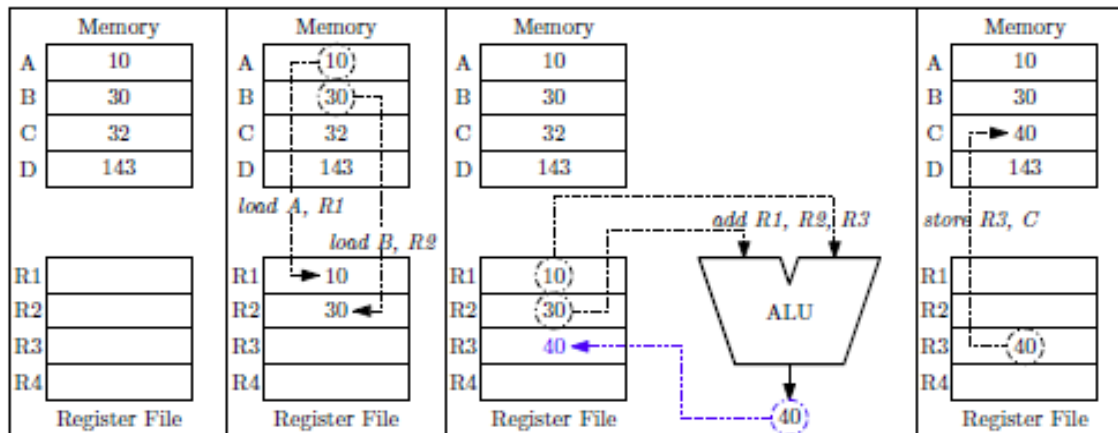


Figure 1.14: Execute $C = A + B$ in the Register-Register architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

In-class Exercise

- Exercise 1.5 Translate the following expression into assembly language.

$$A = (A+B)-(C+D)-E$$

Here, A, B, C, D, and E are memory locations.

Register-Memory Architecture

- Register-memory architecture allows at most one operand to be a memory location.
- This architecture can be found in Intel processors.

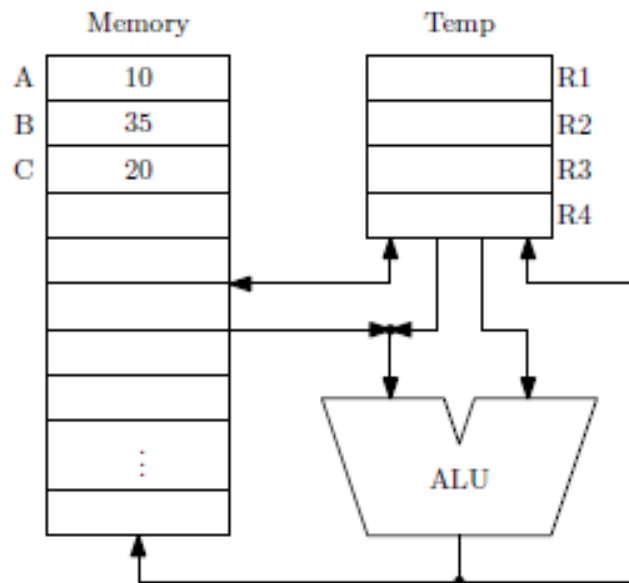


Figure 1.15: Register-Memory architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

Register-Memory Architecture

- Example 1.4 An ISA with the register-memory architecture has eight types of instructions.
 1. load M,R = to read a value of location M and save it to the register R.
 2. store R,M = to write the value of register R to the location M in the main memory.
 3. add R1,R2 = to add register R1 and R2, and save the result in register R2.
 4. sub R1,R2 = to subtract register R1 from register R2, and save the result in register R2.

Register-Memory Architecture

- Example 1.4 An ISA with the register-memory architecture has eight types of instructions.
5. `add R1,M` = to add register R1 and the value in the location M and save the result in the location M.
 6. `sub R1,M` = to subtract register R1 from the value in the location M and save the result in the location M.
 7. `add M,R1` = to add register R1 and the value in the location M and save the result in the register R1.
 8. `sub M,R1` = to subtract the value in the location M from register R1 and save the result in the register R1.

Register-Memory Architecture

- An expression $C = A + B$ can be translated into assembly instruction as:

```
load A,R1  
add B,R1  
store R1,C
```

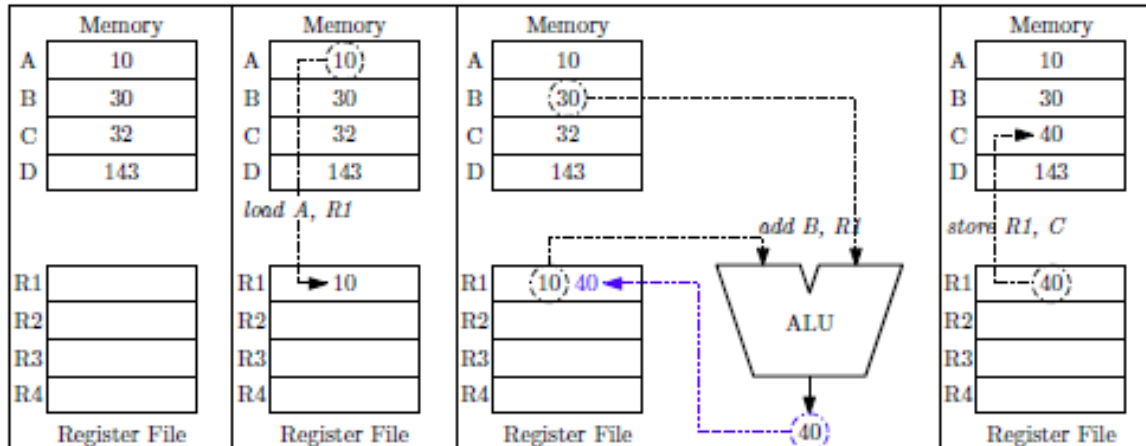


Figure 1.16: Execute $C = A + B$ in the Register-Memory architecture

From C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017, Chapter 1.

In-class Exercise

- Exercise 1.6 Translate the following expression into assembly language.

$$A = (A+B)-(C+D)-E$$

Here, A, B, C, D, and E are memory locations.

References

1. C.Nattee, Lecture Note of CSS224 Computer Architectures, 2017.
2. D.A.Patterson and J.L.Hennessy, Computer Organization and Design: The Hardware/Software Interface, 4th edition; ARM edition, Morgan Kaufmann, ISBN:978-81-312-2274-4.
3. R.E. Bryant and D.R. O'Halloron, Computer Systems: A Programmer's Perspective, 2nd edition, 2011, Pearson/Prentice Hall, ISBN:978-0-13-610804-7.
4. William Stalling, Computer Organization and Architecture Designing for Performance tenth edition, Pearson, ISBN:978-1-292-09685-8.
5. Linda Null and Julia Lobur. The Essentials of Computer Organization and Architecture. Enhanced 4nd Edition.